Complete validation for the projective string-marking theory

The complete diagnostic program is printed below. Its finite samples and false-weakening controls supplement the written uniform proofs. The full program is charged in the separate required-plus-validation sensitivity.

```
# Validation only: finite samples of uniform written proofs and
false-weakening controls.
# Required finite conclusions are separately in required_finite.g.
# Exact bounded projective-line models; uniform proofs are in the
manuscript.
Need258:=function(b,s) if not b then Error(s);fi;end;
Projectivity258:=function(M,p)
 local out,x,den,y;
 out:=[];
 for x in [0..p] do
 if x=p then
 if M[2][1] mod p=0 then y:=p;else y:=
 (M[1][1]*PowerModInt(M[2][1],-1,p)) mod p;fi;
 else den:=(M[2][1]*x+M[2][2]) mod p;
 if den=0 then y:=p;else y:=((M[1][1]*x+M[1][2])*PowerModInt(den,-1,p))
 mod p;fi;
 fi;Add(out,y+1);od;return PermList(out);
end;
String258:=function(rr)
 local i,j;
 if Length(Set(rr))<>Length(rr) or not ForAll(rr,x->Order(x)=2) then
 return false;fi;
 for i in [1..Length(rr)] do for j in [i+1..Length(rr)] do
 if j>i+1 and rr[i]*rr[j]<>rr[j]*rr[i] then return false;fi;
 od;od;return true;
end;
IP258:=function(G,rr)
 local ss,hs,i,j;
 ss:=Combinations([1..Length(rr)]);hs:=List(ss,s->Subgroup(G,rr{s}));
 for i in [1..Length(ss)] do for j in [1..Length(ss)] do
 if Intersection(hs[i],hs[j])<>hs[Position(ss,Intersection(ss[i],ss[j]))]
 then return false;fi;
 od;od;return true;
end;
RunValidation:=function()
 local p,rr,G,orders,a,b,c,C,src,changed,H,subs,inv,x,y,z,triple,D,cent,e
 xt,tgt,ms,dual,ds,i,j;
 for p in [13,37,61] do
 ms:=[[[0,1],[1,0]],[[1,0],[1,-1]],[[-1,0],[0,1]],[[1,-2],[2,-1]]];
 Need258(ForAll(ms,M->PowerModInt(DeterminantMat(M),(p-1)/2,p)=1),"consta
 nt determinant-square membership");
 rr:=List(ms,M->Projectivity258(M,p));
 G:=Group(rr);Need258(String258(rr),"constant string matrices");
 Need258(Size(G)=p*(p^2-1)/2 and IsSimpleGroup(G),"constant PSL image");
 Need258(Size(Subgroup(G,rr{[1..3]}))=Size(G) and
 Size(Subgroup(G,rr{[2..4]}))=Size(G),"two full ends");
 Need258(Order(rr[1]*rr[2])=3 and Order(rr[2]*rr[3])=p,"source3,p type");
 a:=First([1..p-1],x->x^2 mod
 p=p-1);C:=Projectivity258([[-a,0],[0,1]],p);
 src:=List([[[0,1],[-1,0]],[[a,0],[1,-a]],[[-a,0],[0,a]]],M->Projectivity
 258(M,p));
 # GAP right-action permutations reverse composition of the displayed
 column matrices.
```

```
Need258(List(src,s->s^C)=rr{[1..3]},"literal source basis-change
adapter");
Print("CONSTANT_FAMILY ",p," ORDER ",Size(G)," TYPE
",List([1..3],i->Order(rr[i]*rr[i+1]))," SOURCE_CONJUGACY_PASS\n");
od;
# All triples of each chosen maximal subgroup are searched without an IP
input.
for ds in [[41,60,5,3],[47,24,3,4],[73,24,3,4]] do
p:=ds[1];G:=PSL(2,p);
subs:=MaximalSubgroupClassReps(G);
if ds[2]=60 then tgt:=AlternatingGroup(5);else
tgt:=SymmetricGroup(4);fi;
H:=First(subs,x->Size(x)=ds[2] and IsomorphismGroups(x,tgt)<>fail);
Need258(H<>fail,"retained maximal subgroup exists in diagnostic field");
inv:=Filtered(Elements(H),x->Order(x)=2);triple:=fail;
for x in inv do for y in inv do for z in inv do
if Length(Set([x,y,z]))=3 and x*z=z*x and Order(x*y)=ds[3] and
Order(y*z)=ds[4] and Subgroup(H,[x,y,z])=H then
triple:=[x,y,z];break;fi;
od;if triple<>fail then break;fi;od;if triple<>fail then break;fi;od;
Need258(triple<>fail,"source local marking
exists");D:=Subgroup(H,triple{[1,2]});cent:=Centralizer(G,D);
Need258(Size(cent)=2 and Size(Centralizer(H,D))=1,"halfturn leaves
facet");
ext:=First(Elements(cent),x->Order(x)=2);rr:=Concatenation(triple,[ext])
;
Need258(String258(rr) and Subgroup(G,rr)=G and
Subgroup(G,rr{[2..4]})=G,"maximal facet extension");
Print("PROPER_END_FAMILY ",p," FACET ",Size(H)," FULL ",Size(G)," TYPE
",List([1..3],i->Order(rr[i]*rr[i+1]))," PASS\n");
od;
rr:=[(3,9)(4,10)(5,11)(6,12)(7,13)(8,14),(1,4)(2,14)(3,9)(5,13)(6,11)(7,
12),
(1,2)(3,7)(4,6)(8,14)(9,13)(10,12),(1,2)(4,14)(5,13)(6,12)(7,11)(8,10)];
G:=PSL(2,13);dual:=(1,3)(2,9)(4,7)(5,8)(6,12)(10,11)(13,14);
ext:=Concatenation(rr,[()]);
Need258(Length(Set(ext))=5 and Group(ext)=G and
ForAll(ext,x->x*x=()),"rankfive identity-marker countermodel");
for i in [1..5] do for j in [1..5] do if j>i+1 then
Need258(ext[i]*ext[j]=ext[j]*ext[i],"rankfive string
commutations");fi;od;od;
Need258(not String258(ext),"nonidentity condition is essential");
Print("RANK_BOUND_ALLOW_IDENTITY_FALSE_WEAKENING_RANK5_REJECTED\n");
# Omitting IP in an arbitrary group really enlarges the local class.
G:=DihedralGroup(IsPermGroup,12);inv:=Filtered(Elements(G),x->Order(x)=2
);rr:=fail;
for a in inv do for b in inv do for c in inv do
if Length(Set([a,b,c]))=3 and a*c=c*a and Subgroup(G,[a,b,c])=G
and Order(a*b)>2 and Order(b*c)>2 and not IP258(G,[a,b,c]) then
rr:=[a,b,c];break;fi;
od;if rr<>fail then break;fi;od;if rr<>fail then break;fi;od;
Need258(rr<>fail,"no blanket sggi implies C assertion");
Print("DIHEDRAL_NON_C_SGGI_CONTROL TYPE
",List([1,2],i->Order(rr[i]*rr[i+1])),"\n");
# A centre-free group with the common-centralizer condition can have
five
# distinct involutory generators if the string relations are omitted.
G:=AlternatingGroup(5);
```

```
 rr:=[(1,2)(3,4),(1,3)(2,4),(1,2)(3,5),(1,3)(2,5),(1,4)(2,5)];
 Need258(Length(Set(rr))=5 and Group(rr)=G,"no-string five-generator
 control");
 inv:=Filtered(Elements(G),x->Order(x)=2);
 for a in inv do for b in inv do if a*b<>b*a then
 Need258(Size(Centralizer(G,Subgroup(G,[a,b])))<=2,"pair-centralizer
 premise in control");
 fi;od;od;
 Need258(rr[1]*rr[3]<>rr[3]*rr[1],"actual omitted string relation");
 Print("RANK_BOUND_NO_STRING_FALSE_WEAKENING_A5_FIVE_MARKERS_REJECTED\n")
 ;
 Print("TOPIC_E_VALIDATION_PASS\n");
end;
RunValidation();QUIT;
```